

Fundamentos de la programación 2

Grado en Desarrollo de Videojuegos

Convocatoria ordinaria 2026

Indicaciones generales:

- Se entregarán únicamente los archivos fuente (`Program.cs`, `Tablero.cs`, `Coor.cs` y `Lista.cs`). Se proporciona una **plantilla** con el esquema del programa y algunos métodos ya implementados, así como las clases `Coor` y `Lista` (vistas en clase).
 - En todos los archivos fuente, las líneas 1 y 2 serán comentarios de la forma:
`// Nombre Apellido1 Apellido2`
`// Laboratorio, puesto`
 - Lee atentamente el enunciado e implementa el programa tal como se pide, con las clases, métodos, parámetros y requisitos que se especifican. No puede modificarse la representación propuesta, ni alterar los parámetros de los métodos especificados, a excepción de la forma de paso (`out`, `ref`, ...), que debe determinar el alumno. Pueden implementarse todos los métodos adicionales que se consideren oportunos, especificando claramente su cometido, parámetros, etc.
 - El programa, además de correcto, debe estar bien estructurado y comentado. Se valorarán la claridad, la concisión y la eficiencia.
 - La **entrega**: se realizará a través del servidor FTP de los laboratorios. Se entregará un único archivo de la forma **Apellido1_Apellido2_Nombre.zip** con los archivos fuente pedidos.
-

Implementaremos un programa para jugar al *Sudoku*. El juego consta de una cuadrícula de 9×9 celdas, dividida en subcuadrículas de 3×3 celdas. Inicialmente algunas celdas ya están rellenas con números del 1 al 9 y otras vacías. A continuación se muestra un ejemplo:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

El objetivo es rellenar todas las celdas vacías con números del 1 al 9, de manera que ninguno se repita en la misma fila, ni en la misma columna, ni en la misma subcuadrícula (de modo equivalente: cada fila, cada columna y cada subcuadrícula contendrá todos los números del 1 al 9 sin repetición).

La clase `Tablero` tiene el siguiente aspecto.

```
public class Tablero {  
    struct Casilla { // struct para representar cada casillas del tablero  
        public int num; // número en la casilla  
        public bool fija; // casilla fija (true) o no fija (false)  
    }  
    Casilla [,] mat; // matriz de casillas del sudoku  
    Coor cursor; // posición actual del cursor en el tablero  
    ...  
}
```

La matriz `mat` representa las casillas, cada una con un número `num` (entre 1 y 9 si está rellena, y 0 si está vacía) y un booleano `fija` que indica si el número está fijado en el sudoku original. Por ejemplo, en el sudoku anterior, la casilla `[0,0]` tendrá `num=5` y `fija=true` (no puede modificarse). La casilla `[0,2]` tendrá `num=0` (vacía) y `fija=false` (puede rellenarse `num` durante el juego). El campo `cursor`, de tipo `Coor` denota la posición activa para poner números.

En la clase `Tablero` implementaremos los siguientes métodos:

- **[0.5 pt]** `public Tablero(int [,] sud)`: constructora que crea la matriz `mat` con 9×9 casillas y las rellena con los números de la matriz de entrada `sud`. Para cada casilla, el campo `fija` se asigna a `true` si el número es distinto de 0, y a `false` en caso contrario. Además crea la coordenada `(0,0)` para el cursor.
- **[0.5 pt]** `public void Render()`: muestra en pantalla el estado del sudoku. Para facilitar la lectura dejaremos un **espacio en blanco entre las celdas**. Las celdas fijas se escriben en amarillo con `Console.ForegroundColor = ConsoleColor.Yellow`; y las otras en verde (`Green`). Utilizaremos `'.'` para las celdas vacías (las que contienen un 0). El cursor se situará en la posición correspondiente con `Console.SetCursorPosition(left,top)`. Por ejemplo, el sudoku de arriba quedaría así (el cursor en `(1,7)`):

5	.	.	6	.	.	9	.	.
.	.	2	.	.	5	.	8	.
1	9	.	.	.	2	.	.	.
.	.	.	7	.	.	4	.	3
.	.	6	8	.	.	7	.	.
.	1	.	.	2	.	8	5	6
9	.	1	.	3	7	.	8	4
.	8	.	.	.	9	6	.	.
3	.	5	.	.	.	1	.	9

- **[0.5 pt]** `public void MueveCursor(char d)`: mueve la posición del `cursor` hacia arriba, abajo, izquierda o derecha, en función del valor del argumento `d` (puede ser `'u'`, `'d'`, `'l'`, `'r'`). Si la posición obtenida se saliese de los límites del tablero, el método no hace nada.
- **[0.5 pt]** `private void Esquina(int fil, int col)`: modifica la posición `(fil,col)` para convertirla en la esquina superior izquierda de la submatriz a la que pertenece. Es decir, calcula la esquina a partir de la cual hay que comprobar la submatriz correspondiente. Por ejemplo, transformará `(1,2)` en `(0,0)`, y `(3,5)` en `(3,3)`.
- **[1 pt]** `private bool [] Posibles()`: devuelve los posibles valores que pueden colocarse en la posición actual del cursor. Para ello genera un vector de booleanos `pos` de tamaño 10 (indexado de 0 a 9), de modo que `pos[i]` es `true` si el número `i` puede ir en la posición del cursor y `false` en caso contrario.

Para ello se crea el vector `pos` con todas sus componentes a `true` y después, para cada valor `i` en la misma fila, columna o submatriz que el cursor, se asigna `pos[i]=false` (será útil el método `Esquina`). Así, para la posición `[0,2]` del sudoku del ejemplo, `pos` tendrá `true` para las posiciones 0, 3, 4, 7, 8, que son los números posibles en esa posición.

- **[0.5 pt]** `private bool Comprueba(int num)`: comprueba si es posible colocar el número `num` en la posición actual del `cursor`. Para ello utiliza el método anterior `Posibles` y comprueba si `num` está entre los candidatos posibles.

- [1 pt] `public void PonNumero(int num)`: pone el número `num` en la posición actual del cursor. Para ello comprueba previamente que dicho número está en el rango 0..9 y además que puede colocarse en esa posición, utilizando el método `Comprueba`.

Si no puede ponerse `num` lanzará una excepción que se capturará cuando se invoque al método para escribir un mensaje de error en pantalla (se verá más adelante).

- [0.5 pt] `public bool FinJuego()`: devuelve `true` si el sudoku está terminado (es decir, si todas las celdas están rellenas); `false` en caso contrario.
- [1 pt] `public Lista DamePosibles()`: utilizando el método `Posibles` y la clase `Lista`, devuelve una lista con los posibles valores para la posición del cursor. Para el sudoku del ejemplo, para [0,1] devolverá una lista con 3, 4 y 7. Para una posición con valor fijo dado, devolverá dicho valor. Por ejemplo, para la posición [0,0] devolverá una lista con el 5.

En `Program.cs` se da implementado el método `LeeInput` e implementaremos los siguientes:

- [1 pt] `static void ProcesaInput(char c, Tablero t)`: actúa sobre el tablero en función del carácter que reciba como argumento. Responderá a los siguientes caracteres:
 - 'l', 'r', 'u', 'd': utiliza el método `MueveCursor` de `Tablero` para mover el cursor.
 - 'p': muestra en pantalla los posibles números que pueden colocarse en la posición actual del cursor, utilizando los métodos implementados en la clase `Tablero`.
 - '0'..'9': coloca el número dado en la posición actual del cursor utilizando el método `PonNumero` de `Tablero`. Debe capturarse la posible excepción del método `PonNumero` y dar un mensaje de error correspondiente en ese caso. Recordemos que se puede convertir un carácter '0'..'9' en entero con la instrucción `int i = (int)(c-'0');`
- [1 pt] `public static void Main()`: crea un nuevo tablero, lee el sudoku del archivo proporcionado `sudoku` y lo muestra en pantalla. Después implementa el bucle principal (**por este orden**): lee el input de usuario, lo procesa y dibuja el tablero en cada iteración. Terminará cuando el sudoku esté completado o bien cuando el usuario pulse 'q'.

Para leer el sudoku de un archivo, se implementará en `Program.cs` el siguiente método:

- [1 pt] `static int[,] Lee(string file)`: lee el archivo `file` y devuelve una matriz de enteros con el contenido del sudoku. El formato del archivo puede verse en el archivo `sudoku` proporcionado, que corresponde al sudoku del ejemplo. Recordemos que para trocear una línea de texto `s` en un array de strings utilizando el espacio como separador, se puede utilizar la instrucción: `s.Split(' ',StringSplitOptions.RemoveEmptyEntries)`

Ahora extenderemos el método `Main` para que el usuario pueda utilizar el sudoku de ejemplo, o bien, leer uno desde un archivo que se solicitará al usuario.

A continuación permitiremos al usuario consultar los posibles valores en la posición actual del cursor. Para ello, primero extenderemos la clase `Lista`:

- [1 pt] `public int Cuenta()`: cuenta el número de elementos de la lista **de manera recursiva**, utilizando un método auxiliar con los parámetros necesarios. ¿Puedes hacer una versión recursiva final? **Nota**: las versiones iterativas (con bucles) no puntuarán nada.

Por último extenderemos el método `ProcesaInput` en `Program.cs` para que al pulsar 'p' muestre en pantalla la cantidad de números posibles en la posición actual del cursor, así como los números en cuestión, utilizando los métodos `DamePosibles`, `Cuenta` (así como el método `ToString` de la clase `Lista` ya implementado). Por ejemplo, si el cursor está en la posición `[0,1]`, mostrará el mensaje (en una fila por debajo del sudoku):

3 números posibles: 3 4 7